23 September 2025

0001 0011 1010 1111 0001 → 0x13AF1

1 3 A F 1

chr → 1 byte
short → 2 byte
integer → dependent upon architecture
long int → 4 bytes
float → 4 bytes
double → 8 bytes

MSB — n-1 bits for signed

0 +ve
1 -ve

Unsigned int : 2^{31} ——— 2^0

Limit: $2^{32} - 1$ unsigned long int

For signed

$$\frac{2^{32}}{2} = 2^{31}$$

N- bit number

2^n Patterens

limit/range $0 - 2^n - 1$

if

$-2^{N-1} - 0 - 2^{N-1}$ then 2^{N+1}

→ so have to reduce one number from positive side bcs zero takes place of one number and there is no such thing as -ve zero so

$-2^{31} - 2^{30}$

for signed short

$-2^{15} \text{ — } 2^{14}$

for unsigned

$0 \text{ — } \boxed{2^{16}-1}$

for char

signed: $-2^{7} \text{ — } 2^{6}$

unsigned $\boxed{0 \text{ — } 2^{8}-1}$

Type Casting

```
float x = 3.2;
int y;
y = ((int) x) + 2
```

Functions

output data type ↑ ↗ input data types

```
int main (void)
{

}
return ___
```

→ indentation doesn't matter in C
the curly brackets tell us

Shifting

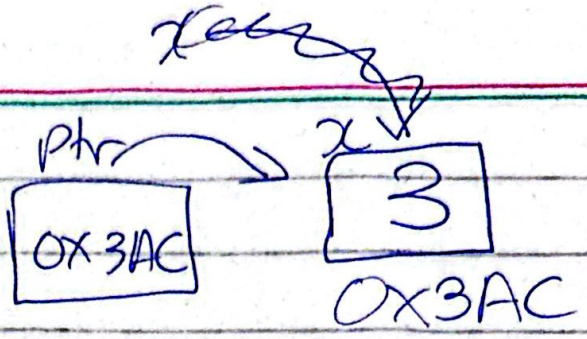
011011 >> 000110
011011 << 101100

Pointer

→ Stores address
→ points to data

int *ptr يحتوي على عنوان بيانات من نوع datatype

→ the data type of pointer is unsigned integer.



int x = 3;

ptr = &x;

to access this value

*ptr = 3 changes value

ptr = 0X3AC changes address

*ptr++ ⇒ 4

ptr++; ⇒ 3B0

4 bytes added to address

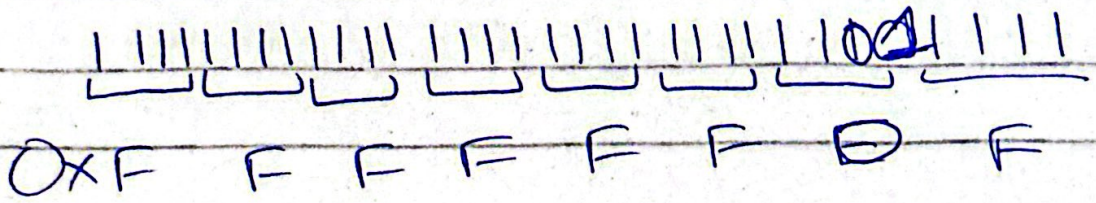
Turn 5th bit of led

```
int bit_led(int *led) {        // if want to do through pointers
    led = *led | 0x00000020;
    return led
}

int main (void){
    int x;
    x = 1567;
    bitset(&x);                // can skip &
}
```

```
int bit_cken (led) {
    led = led & 0xFFFFFFBF;
    return led;
}
```

0xFFFFFFBF: 6 of previous number



OFF ~~AND~~ → led & ~BITS

BitS = 0x00000020 → ON → led |

```
int bit_set(int led, int bitno){
    led = led | (0x00000001 << bit_no)
    return led;
}
```

led = led & ~(0x00000001 << bit_no)

constant and volatile

Always known by compiler

data type and type specifier

type qualifier

- const
- volatile (Register) ≤ unsigned integer

Register Address :

0X40004400

MACROS

#define

- Object-like MACROS
- function like macros

#define HELLO 5 → object like macro

#define SUM(A+B) A+B → function like macro

#include < .h> } globally recognized header files

#include " .h" } software dependent on self written

#define GPIO_DIR 0x40004400

Register address but must be address

dereference.

#define GPIO_DIR *((volatile unsigned int *) 0x40004400))